Building Our Digital World: Computer Systems and Architecture
COMP1860

School of Computer Science, University of Leeds

Activity Sheet 1.2

1 Binary & Hexadecimal

1.1 Conversion

1. Convert the following denary (base 10) numbers to binary (base 2):
   (a) 3
   Solution: 101
   (b) 12
   Solution: 1100
   (c) 126
   Solution: 1111110
   (d) 128
   Solution: 10000000
   (e) 235
   Solution: 11101011
2. Convert the following binary numbers to denary:
   (a) 10
   Solution: 2
   (b) 101
   Solution: 5
   (c) 1011
   Solution: 11
   (d) 11010
   Solution: 26
   (e) 11100101
   Solution: 229
3. Convert the binary number **10110011** to hexadecimal.
   Solution: Split into groups of 4 bits (one hexadecimal character). 1011 is 11 in decimal, which is B in hexadecimal. 0011 is 3 in decimal, which is 3 in hexadecimal. Hence, B3.
4. Convert the hexadecimal number **A5** to decimal.
   Solution: Remember each hex digit is a power of 16. Hence A5 is $\mathrm{A}\times 16^1 + 5 \times 16^0$. A is 10 in decimal, so the result is $10 \times 16 + 5 \times 1 = 165$.
5. Convert the decimal number **270** to hexadecimal.
   Solution: 256 is 100. This leaves remainder 14 which is E in hexadecimal. Hence, 10E.

1.2 Binary Arithmetic

1. Add the following binary numbers using binary addition:

   (a) 11 and 10
   Solution:

$$\begin{array}{r} 11 \\ +\,10 \\ \hline 101 \end{array}$$

   (b) 100 and 110
   Solution:

$$\begin{array}{r} 100 \\ +\,110 \\ \hline 1010 \end{array}$$

   (c) 1101 and 1001
   Solution:

$$\begin{array}{r} 1101 \\ +\,1001 \\ \hline 10110 \end{array}$$

   (d) 1101 and 10101
   Solution:

$$\begin{array}{r} 1101 \\ +\,10101 \\ \hline 100010 \end{array}$$

   (e) 10110010 and 10001101
   Solution:

$$\begin{array}{r} 10110010 \\ +\,10001101 \\ \hline 100111111 \end{array}$$

2. Multiply the following binary numbers using binary long multiplication:

   (a) 11 and 10
   Solution:

$$\begin{array}{r} 11 \\ \times\,10 \\ \hline 00 \\ +\,11 \\ \hline 110 \end{array}$$

   (b) 111 and 10
   Solution:

$$\begin{array}{r} 111 \\ \times\,10 \\ \hline 000 \\ +\,111 \\ \hline 1110 \end{array}$$

(c) 1101 and 110
Solution:

$$\begin{array}{r} 1101 \\ \times\ 110 \\ \hline 0000 \\ 1101 \\ +\ 1101 \\ \hline 1001110 \end{array}$$

1.3 Binary Fractions

(a) Convert the following binary fractions to decimal fractions:
   (a) 1.1
   Solution: .1 is $2^{-1} = 0.5$. Hence, 1.5.
   (b) 10.01
   Solution: 10 is 2, .01 is $2^{-2} = 0.25$. Hence, 2.25.
   (c) 111.11
   Solution: 111 is 7, .11 is $2^{-1} + 2^{-2} = 0.75$. Hence, 7.75.
   (d) 101.101
   Solution: 101 is 5, .101 is $2^{-1} + 2^{-3} = 0.625$. Hence, 5.625.
   (e) 1011.111
   Solution: 1011 is 11, .111 is $2^{-1} + 2^{-2} + 2^{-3} = 0.875$. Hence, 11.875.

(b) Convert the following decimal fractions to binary fractions:
   (a) 1.5
   Solution: 1.1. See above.
   (b) 22.25
   Solution: 10110 is 22. .01 is 0.25. Hence 10110.01.
   (c) 23.75
   Solution: 10111 is 23. .11 is 0.75. Hence 10111.11.
   (d) 25.125
   Solution: 11001 is 25. .001 is 0.125. Hence 11001.001.

(c) Can the decimal floating-point number 3.2 be converted to binary exactly? *Reflection:* If not, how might this be approximated in binary?
Solution: No, it cannot be represented exactly. It can be represented as 101.0011^*, i.e. 0011 recurs infinitely. 0.0011 is 0.1875. 0.00110011 is 0.19921875, etc. The fractional part gets closer to 0.2 as the sequence grows. However, we can't store infinite sequences in a computer, so this would have to be rounded, hence it is an approximation.

2 Integer & Floating-point Representation

2.1 Integer Representation

(a) What is the binary representation of **125** in 4-bit binary coded decimal format?
Solution: Convert each decimal digit to a 4-bit binary number. 1 is 0001, 2 is 0010, 5 is 0101. Hence, **0001 0010 0101**.

(b) What is the binary representation of **125** in 8-bit two's complement integer format?
Solution: 125 is **0111 1101**. We have a leading zero to make an 8-bit value.

(c) What is the binary representation of **-125** in 8-bit two's complement integer format?
Solution: First convert 125 to binary, 0111 1101. Next, flip all digits to get 1's complement 1000 0010. Now add 1, to get **1000 0011**.

(d) For the decimal numbers 72 and -56:

(a) Convert each number to 8-bit two's complement integer format.
Solution: 72 is 0100 1000. -56 is 1100 1000.

(b) Add the two resulting numbers using binary addition. **Note:** Remember to adjust the result if it is longer than 8 bits.
Solution:

$$\begin{array}{r} 01001000 \\ +\,11001000 \\ \hline 100010000 \end{array}$$

As the signs of the two operands differ, then no overflow has occurred. Therefore, we ignore the carry (leading 1). Hence, 00010000.

(c) Convert your result back to decimal. Is this correct?
Solution: 00010000 is 16, which is the result of $72 - 56$

2.2 IEEE 754 Floating-point Format

(a) Convert the following decimal numbers to single precision 32-bit IEEE 754 floating point format:

(a) 235.5
Solution: 235 is 11101011. 0.5 is 0.1. Hence 11101011.1. Converted to scientific format this is 1.1101011×2^7. Hence, the mantissa is 110 1011 0000 0000 0000 0000 (the binary fractional part postpended by zeroes to make a 23 bit value). The exponent is 7. The biased exponent is 127 + 7, hence 134, which is 1000 0110. This is positive so the sign bit is 0. Therefore, it is 0100 0011 0110 1011 1000 0000 0000 0000.

(b) 1024.25
Solution: 1024 is 10000000000. 0.25 is 0.01. Hence 10000000000.01. Converted to scientific format this is $1.000000000001 \times 2^{10}$. Hence, the mantissa is 000 0000 0000 1000 0000 0000. The exponent is 10. The biased exponent is 127 + 10, hence 137, which is 1000 1001. This is positive so the sign bit is 0. Therefore, it is 0100 0100 1000 0000 0000 1000 0000 0000.

(c) 12345.375
Solution: 12345 is 11000000111001. 0.375 is 0.011. Hence 11000000111001.011. Converted to scientific format this is $1.1000000111001011 \times 2^{13}$. Hence, the mantissa is 100 0000 1110 0101 1000 0000. The exponent is 13. The biased exponent is 127 + 13, hence 140, which is 1000 1100. This is positive so the sign bit is 0. Therefore, it is 0100 0110 0100 0000 1110 0101 1000 0000.